

448. 找到所有数组中消失的数字

已解答

简单 相关标签 相关企业 提示 Az

给你一个含 n 个整数的数组 `nums`，其中 `nums[i]` 在区间 $[1, n]$ 内。请你找出所有在 $[1, n]$ 范围内但没有出现在 `nums` 中的数字，并以数组的形式返回结果。

示例 1:

输入: `nums = [4,3,2,7,8,2,3,1]`
输出: `[5,6]`

示例 2:

输入: `nums = [1,1]`
输出: `[2]`

提示:

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> findDisappearedNumbers(vector<int>& nums) {
4         int n = nums.size();
5         vector<int> count(n+1);
6         for(auto num:nums){
7             count[num]++;
8         }
9         vector<int> ans;
10        for(int i =1;i<=n;i++){
11            if(count[i] == 0){
12                ans.push_back(i);
13            }
14        }
15        return ans;
16    }
17};
```

1365. 有多少小于当前数字的数字

已解答

简单 相关标签 相关企业 提示 Az

给你一个数组 `nums`，对于其中每个元素 `nums[i]`，请你统计数组中比它小的所有数字的数目。

换言之，对于每个 `nums[i]` 你必须计算出有效的 `j` 的数量，其中 `j` 满足 `j != i` 且 `nums[j] < nums[i]`。

以数组形式返回答案。

示例 1:

输入: `nums = [8,1,2,2,3]`
输出: `[4,0,1,1,3]`
解释:

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> smallerNumbersThanCurrent(vector<int>& nums) {
4         int len = nums.size();
5         vector<int> ans(len);
6         for(int i = 0;i<len;i++){
7             for(int j = 0;j<len;j++){
8                 if(j == i)continue;
9                 if(nums[i] > nums[j])ans[i]++;
10            }
11        }
12        return ans;
13    }
14};
```

645. 错误的集合

已解答

简单 相关标签 相关企业 Az

集合 S 包含从 1 到 n 的整数。不幸的是，因为数据错误，导致集合里面某一个数字复制成了集合里面的另外一个数字的值，导致集合 **丢失了一个数字** 并且 **有一个数字重复**。

给定一个数组 `nums` 代表了该集合发生错误后的结果。

请你找出重复出现的整数，再找到丢失的整数，将它们以数组的形式返回。

示例 1:

输入: `nums = [1,2,2,4]`
输出: `[2,3]`

示例 2:

输入: `nums = [1,1]`
输出: `[1,2]`

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> findErrorNums(vector<int>& nums) {
4         int first=-1,second=-1;
5         int n = nums.size();
6         vector<int> v(n+1);
7         for(auto num: nums){
8             v[num]++;
9         }
10        for(int i = 1;i<=n;i++){
11            if(v[i] == 2){
12                first = i;
13            }
14            if(v[i] == 0){
15                second = i;
16            }
17            if(first != -1 && second != -1)break;
18        }
19        return {first,second};
20    }
21};
```

485. 最大连续 1 的个数

已解答

简单 相关标签 相关企业 提示 Az

给定一个二进制数组 `nums`，计算其中最大连续 `1` 的个数。

示例 1:

输入: `nums = [1,1,0,1,1,1]`
输出: `3`
解释: 开头的两位和最后的三位都是连续 `1`，所以最大连续 `1` 的个数是 `3`。

示例 2:

输入: `nums = [1,0,1,1,1,0,1]`
输出: `2`

提示:

- `1 <= nums.length <= 10^5`

C++ 智能模式

```
1 class Solution {
2 public:
3     int findMaxConsecutiveOnes(vector<int>& nums) {
4         int ans = INT_MIN;
5         int count = 0;
6         for(auto num:nums){
7             if(num == 0){
8                 ans = max(ans,count);
9                 count = 0;
10            }
11            if(num == 1){
12                count++;
13            }
14        }
15        ans = max(ans,count);
16        return ans;
17    }
18};
```

已存储

1470. 重新排列数组

已解答

简单 相关标签 相关企业 提示 Aa

给你一个数组 `nums`，数组中有 `2n` 个元素，按 `[x1, x2, ..., xn, y1, y2, ..., yn]` 的格式排列。

请你将数组按 `[x1, y1, x2, y2, ..., xn, yn]` 格式重新排列，返回重排后的数组。

示例 1:

输入: `nums = [2,5,1,3,4,7]`, `n = 3`

输出: `[2,3,5,4,1,7]`

解释: 由于 `x1=2`, `x2=5`, `x3=1`, `y1=3`, `y2=4`, `y3=7`，所以答案为 `[2,3,5,4,1,7]`

示例 2:

输入: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`

输出: `[1,4,2,3,3,2,4,1]`

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> shuffle(vector<int>& nums, int n) {
4         int len = n*2;
5         vector<int>ans(2*n);
6         int ch1 = 0, ch2 = n;
7         for(int i = 0; i < len; i++){
8             if(i%2==0)ans[i] = nums[ch1++];
9             if(i%2!=0)ans[i] = nums[ch2++];
10        }
11        return ans;
12    }
13};
```

1929. 数组串联

已解答

简单 相关标签 相关企业 提示 Aa

给你一个长度为 `n` 的整数数组 `nums`。请你构建一个长度为 `2n` 的答案数组 `ans`，数组下标从 `0` 开始计数，对于所有 `0 ≤ i < n` 的 `i`，满足下述所有要求

- `ans[i] == nums[i]`
- `ans[i + n] == nums[i]`

具体而言，`ans` 由两个 `nums` 数组 串联 形成。

返回数组 `ans`。

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> getConcatenation(vector<int>& nums) {
4         int n = nums.size();
5         vector<int>ans(2*n);
6         for(int i = 0; i < n; i++){
7             ans[i] = nums[i];
8             ans[i+n] = nums[i];
9         }
10        return ans;
11    }
12};
```

LCR 019. 验证回文串 II

已解答

简单 相关标签 相关企业 Aa

给定一个非空字符串 `s`，请判断如果最多从字符串中删除一个字符能否得到一个回文字符串。

示例 1:

输入: `s = "aba"`
输出: `true`

示例 2:

输入: `s = "abca"`
输出: `true`
解释: 可以删除 "c" 字符 或者 "b" 字符

示例 3:

输入: `s = "abc"`
输出: `false`

提示:

- `1 ≤ s.length ≤ 105`
- `s` 由小写英文字母组成

C++ 智能模式

```
1 class Solution {
2 public:
3     bool check(vector<char>& str, int l, int r)
4     {
5         while(l < r)
6         {
7             if(str[l] != str[r]) return false;
8             l++; r--;
9         }
10        return true;
11    }
12    bool validPalindrome(string s) {
13        vector<char>str;
14        for(auto ch:s){
15            if(ch>='A' && ch<='Z') ch += 32;
16            if( (ch>='a' && ch<='z') || (ch>='0' && ch<='9') ){
17                str.push_back(ch);
18            }
19        }
20        int i=0, j=str.size()-1;
21        while(i < j){
22            if(str[i] == str[j]){
23                i++; j--;
24            }else{
25                // 不等的时候，两种机会，删i OR 删j，只要一个可行就true
26                return check(str, i+1, j) || check(str, i, j-1);
27            }
28        }
29        return true;
30    }
31};
32
```

LCR 018. 验证回文串

已解答 

简单 相关标签 相关企业 A+

给定一个字符串 `s`，验证 `s` 是否是 **回文串**，只考虑字母和数字字符，可以忽略字母的大小写。

本题中，将空字符串定义为有效的 **回文串**。

示例 1:

输入: `s = "A man, a plan, a canal: Panama"`

输出: `true`

解释: "amanaplanacanalpanama" 是回文串

示例 2:

输入: `s = "race a car"`

输出: `false`

解释: "raceacar" 不是回文串

C++ 智能模式

```
1 class Solution {
2 public:
3     bool isPalindrome(string s) {
4         vector<char>str;
5         for(auto ch:s){
6             if(ch>='A' && ch<='Z'){
7                 ch += 32;
8             }
9             if(ch>='a' && ch<='z' || ch>='0' && ch<='9'){
10                 str.push_back(ch);
11             }
12         }
13         int i=0,j=str.size()-1;
14         while(i<=j){
15             if(str[i] != str[j]){
16                 i++;j--;
17             }else{
18                 return false;
19             }
20         }
21         return true;
22     }
```